

PyTorch 目标检测面试代码修改指南

基于 Faster R-CNN 示例代码，快速改成 Cat/Dog Object Detection 任务

核心目标：面试中不需要从零背完整代码，而是要知道哪些模块必须灵活修改：数据路径、annotation 读取方式、类别数量、label mapping、模型输出头、训练参数、推理可视化和 checkpoint 信息。

1. 你的示例代码整体逻辑

- 当前代码是一个 torchvision Faster R-CNN ResNet50-FPN 目标检测模板，原始任务是 PennFudanPed pedestrian detection。
- 代码从 mask 中生成 bounding boxes，然后训练 Faster R-CNN，并在验证图片上画出预测框。
- 如果面试任务变成猫/狗检测，训练循环和模型主体大部分可以保留，最关键的是替换 Dataset 解析逻辑和类别定义。

模块	当前示例	猫/狗检测时如何改
DATA_ROOT / dataset_dir	下载并读取 PennFudanPed	改成面试给你的 Data 文件夹
PennFudanDataset	读取 PNGImages + PedMasks	根据 XML / YOLO TXT / COCO JSON 读取图片和 bbox
NUM_CLASSES	2 = background + pedestrian	3 = background + cat + dog
labels	全部设为 1	cat=1, dog=2; 0 永远留给 background
build_model()	替换 Faster R-CNN predictor	保留，只把 num_classes 传成 3
train_one_epoch()	检测模型内部返回 loss_dict	基本保留
collate_fn()	处理 variable number of boxes	必须保留
run_inference()	显示 pedestrian	改成 CLASS_NAMES[label]
save_checkpoint()	保存 Pedestrian 类名	保存 cat/dog 类名

2. 面试第一步：先判断数据集 annotation 格式

面试时先说：I will first inspect the dataset structure and annotation format, because the Dataset class depends on whether the labels are VOC XML, YOLO TXT, COCO JSON, or segmentation masks.

```
import os

print(os.listdir("Data"))
print(os.listdir("Data/images")[:5])
# 根据实际情况检查 annotations / labels / masks 文件夹
print(os.listdir("Data/annotations")[:5])
```

- .xml 通常是 VOC annotation；需要解析 xmin/ymin/xmax/ymax。
- .txt 通常是 YOLO annotation；需要把 normalized x_center, y_center, width, height 转换成 xmin/ymin/xmax/ymax。

- .json 可能是 COCO annotation；需要按 image_id 读取 annotations。
- .png mask 说明可以像 PennFudan 那样从 mask 提取 bbox。
- 如果只有 cat/dog 图片标签，没有 bounding box，那是 classification，不是 object detection。

3. 必须修改的全局变量

```
DATA_ROOT = "Data"
OUTPUT_DIR = "outputs_catdog_detection"
CHECKPOINT_PATH = os.path.join(OUTPUT_DIR, "fasterrcnn_catdog.pth")

CLASS_NAMES = ["background", "cat", "dog"]
NUM_CLASSES = len(CLASS_NAMES) # 3: background + cat + dog

BATCH_SIZE = 1 # CPU 推荐 1; GPU 可以 2 或 4
NUM_EPOCHS = 1 # 面试快速验证可设 1-3
LEARNING_RATE = 0.005
NUM_WORKERS = 0 # Windows / Jupyter 中更稳定
SCORE_THRESHOLD = 0.5
TRAIN_RATIO = 0.8
```

关键解释：Faster R-CNN 中 label 0 是 background，所以真正的类别必须从 1 开始。cat=1, dog=2，因此 NUM_CLASSES=3。

4. Dataset 类必须返回的 target 格式

```
target = {
    "boxes": boxes, # FloatTensor[N, 4], format: [xmin, ymin, xmax, ymax]
    "labels": labels, # Int64Tensor[N], e.g. cat=1, dog=2
    "image_id": image_id, # Tensor[1]
    "area": area, # Tensor[N]
    "iscrowd": iscrowd # Int64Tensor[N]
}
```

- boxes 必须是 [xmin, ymin, xmax, ymax]，不是 YOLO 的 [x_center, y_center, width, height]。
- labels 必须是 torch.int64。
- boxes 必须是 torch.float32。
- 每张图片可以有不同数量的 objects，所以 DataLoader 必须用 collate_fn。

5. 如果面试数据是 VOC XML 格式

```
import xml.etree.ElementTree as ET

class CatDogVOCDataset(torch.utils.data.Dataset):
    def __init__(self, root, transforms=None):
        self.root = root
        self.transforms = transforms
        self.img_dir = os.path.join(root, "images")
        self.ann_dir = os.path.join(root, "annotations")
```

```

self.imgs = sorted([
    f for f in os.listdir(self.img_dir)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])
self.label_map = {"cat": 1, "dog": 2}

def __getitem__(self, idx):
    img_name = self.imgs[idx]
    img_path = os.path.join(self.img_dir, img_name)
    xml_name = os.path.splitext(img_name)[0] + ".xml"
    ann_path = os.path.join(self.ann_dir, xml_name)

    img = Image.open(img_path).convert("RGB")
    tree = ET.parse(ann_path)
    xml_root = tree.getroot()

    boxes, labels = [], []
    for obj in xml_root.findall("object"):
        class_name = obj.find("name").text.lower().strip()
        if class_name not in self.label_map:
            continue
        bbox = obj.find("bndbox")
        xmin = float(bbox.find("xmin").text)
        ymin = float(bbox.find("ymin").text)
        xmax = float(bbox.find("xmax").text)
        ymax = float(bbox.find("ymax").text)
        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(self.label_map[class_name])

    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.as_tensor(labels, dtype=torch.int64)
    image_id = torch.tensor([idx])
    area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
    iscrowd = torch.zeros((len(boxes),), dtype=torch.int64)

    target = {"boxes": boxes, "labels": labels, "image_id": image_id,
              "area": area, "iscrowd": iscrowd}

    if self.transforms is not None:
        img, target = self.transforms(img, target)
    return img, target

def __len__(self):
    return len(self.imgs)

```

6. 如果面试数据是 YOLO TXT 格式

```

class CatDogYOLODataset(torch.utils.data.Dataset):
    def __init__(self, root, transforms=None):
        self.root = root
        self.transforms = transforms
        self.img_dir = os.path.join(root, "images")
        self.label_dir = os.path.join(root, "labels")
        self.imgs = sorted([
            f for f in os.listdir(self.img_dir)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ])

    def __getitem__(self, idx):

```

```

img_name = self.imgs[idx]
img_path = os.path.join(self.img_dir, img_name)
label_name = os.path.splitext(img_name)[0] + ".txt"
label_path = os.path.join(self.label_dir, label_name)

img = Image.open(img_path).convert("RGB")
width, height = img.size
boxes, labels = [], []

with open(label_path, "r") as f:
    for line in f.readlines():
        values = line.strip().split()
        if len(values) != 5:
            continue
        class_id, x_center, y_center, box_w, box_h = map(float, values)

        x_center *= width
        y_center *= height
        box_w *= width
        box_h *= height

        xmin = x_center - box_w / 2
        ymin = y_center - box_h / 2
        xmax = x_center + box_w / 2
        ymax = y_center + box_h / 2

        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(int(class_id) + 1) # YOLO 0/1 -> Faster R-CNN 1/2

boxes = torch.as_tensor(boxes, dtype=torch.float32)
labels = torch.as_tensor(labels, dtype=torch.int64)
image_id = torch.tensor([idx])
area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
iscrowd = torch.zeros((len(boxes),), dtype=torch.int64)

target = {"boxes": boxes, "labels": labels, "image_id": image_id,
          "area": area, "iscrowd": iscrowd}

if self.transforms is not None:
    img, target = self.transforms(img, target)
return img, target

def __len__(self):
    return len(self.imgs)

```

7. 模型部分：基本保留，只改 NUM_CLASSES

```

from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

def build_model(num_classes):
    model = fasterrcnn_resnet50_fpn(weights="DEFAULT")
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

model = build_model(num_classes=NUM_CLASSES)
model.to(device)

```

面试解释： I use a pretrained Faster R-CNN ResNet50-FPN model and replace the ROI box predictor so that the final classifier outputs background, cat, and dog.

8. 训练循环：检测模型不需要你手写 criterion

```
loss_dict = model(images, targets)
losses = sum(loss for loss in loss_dict.values())
```

```
optimizer.zero_grad()
losses.backward()
optimizer.step()
```

- torchvision 的 Faster R-CNN 在 training mode 下输入 images + targets，会自动返回 detection loss_dict。
- 常见 loss 包括 loss_classifier、loss_box_reg、loss_objectness、loss_rpn_box_reg。
- 所以不用像分类任务那样手写 nn.CrossEntropyLoss()。

9. DataLoader 中必须保留 collate_fn

```
def collate_fn(batch):
    return tuple(zip(*batch))

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    collate_fn=collate_fn
)
```

原因：目标检测中每张图片的 bounding boxes 数量可能不同，不能像分类任务一样直接 stack 成一个固定形状 tensor。

10. 推理可视化：把 pedestrian 改成 cat/dog

```
CLASS_NAMES = ["background", "cat", "dog"]

label_texts = [
    f"{CLASS_NAMES[label.item()]} {score:.2f}"
    for label, score in zip(labels, scores)
]
```

- 当前示例代码中写死了 pedestrian 文本，猫/狗检测必须改成根据 label 取 CLASS_NAMES。
- score_threshold 可以保持 0.5；如果预测框太少，可以临时降到 0.3 做 debug。

11. Checkpoint：保存正确类别信息

```
torch.save(
    {
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "loss_history": loss_history,
        "num_classes": NUM_CLASSES,
        "class_names": CLASS_NAMES,
    },
    CHECKPOINT_PATH
)
```

12. main() 中最常改的几行

```
dataset_dir = DATA_ROOT

full_dataset = CatDogVOCDataset(
    root=dataset_dir,
    transforms=get_transform(train=True)
)

test_dataset = CatDogVOCDataset(
    root=dataset_dir,
    transforms=get_transform(train=False)
)

model = build_model(num_classes=NUM_CLASSES)
```

如果是 YOLO 格式：把 CatDogVOCDataset 换成 CatDogYOLODataset 即可。其他训练循环、模型构建、optimizer 和 scheduler 基本不用改。

13. 面试时可以直接说的详细计划

1. First, I will inspect the dataset folder and annotation format to decide how to implement the Dataset class.
2. Then I will define the class mapping. For Faster R-CNN, label 0 is background, so cat and dog will be labels 1 and 2.
3. Next, I will implement a custom Dataset that returns image and target, where target contains boxes, labels, image_id, area, and iscrowd.
4. I will use a pretrained Faster R-CNN ResNet50-FPN model and replace the box predictor for three classes.
5. During training, the torchvision detection model returns a dictionary of losses, so I will sum the losses and backpropagate.
6. Finally, I will run inference on validation images, filter predictions by confidence score, and visualize predicted boxes with class names and scores.

14. 常见错误清单

错误	正确做法
cat/dog 任务把 NUM_CLASSES 设置成 2	应设置成 3, 因为包含 background
cat=0, dog=1	应使用 background=0, cat=1, dog=2
YOLO box 不转换就给 Faster R-CNN	必须转换为 [xmin, ymin, xmax, ymax]
删除 collate_fn	必须保留, 因为每张图片 object 数量不同
只返回 image-level class label	目标检测必须返回 bbox + label
把 labels 设成 float	labels 必须是 int64
CPU 上 batch size 太大	CPU 建议 batch_size=1, epoch=1-3 做快速验证

15. 最应该背熟的三段代码

A. 类别定义

```
CLASS_NAMES = ["background", "cat", "dog"]
NUM_CLASSES = len(CLASS_NAMES)
```

B. 模型输出头替换

```
model = fasterrcnn_resnet50_fpn(weights="DEFAULT")
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, NUM_CLASSES)
```

C. 训练 loss 计算

```
loss_dict = model(images, targets)
losses = sum(loss for loss in loss_dict.values())
losses.backward()
optimizer.step()
```